Module	Advanced Topics in Computer Vision and Deep Learning Coursework Assignment: EEEM071
Assessment type	Coursework assignment
Weighting	25%
Deadline	4:00PM, Friday, 1 May 2026 (Week 10)
STUDENT URN	6954049
STUDENT Name	Amna Ahmad

Objectives:

- Learn and become familiar with the process of sequential hyperparameter tuning of an entire deep learning pipeline.
- Develop intuition via experimentation on how to use such model design and parameter tuning to improve the performance of a pipeline.
- Interpret experimental results of deep learning experiments and their implications.

Outcomes:

- Navigate general deep learning codebases and modify according to requirements.
- Empirically suggest reasonable values for hyperparameters and select the ones enabling the best performance.
- Understand the implications of changing the backbone architecture of a pipeline.
- Reason why data augmentation works.
- Understand the implications and importance of selecting the best learning rate and batch size.
- Understand how to report experimental results professionally. This includes using tables, plots, and diagrams to communicate ideas and results.

Vehicle Re-identification

For this coursework assignment, you are encouraged to apply what you have learnt from the module materials and your participation in collaborative learning activities like lab sessions and discussions.

Guidelines

- Submit your assignment as a **Word or PDF document**, along with the supporting evidence as required, via the SurreyLearn submission page before the deadline. Do not submit any other unrelated documents that could flood your submission and mislead the marking process. Only the latest version submitted before the deadline will be assessed if multiple versions exist.
- Label your submission file as: EEEM071-[insert your URN number]-Assignment e.g., EEEM071-1234567-Assignment
- Include a reference list at the end of your assignment, and use the Harvard or IEEE referencing style when citing sources. Please use the selected option consistently.
- **Word limit:** Each section of the report should not exceed **200 words** (excluding tables, plots, and graphs). Penalties of up to 50% may be applied for unnecessarily long reports.
- This assignment requires time and effort. It is recommended that you start working on it early to minimise the possibility of experiencing stress around it and avoid missing the deadline.
- You will receive both an overall mark and written feedback on your assignment.
- Extensions for this assignment will only be granted under exceptional circumstances. Extension requests must be submitted via the Extenuating Circumstances (ECs) process, as tutors are not permitted to approve ad hoc extension requests.

Note:

Please follow Surrey's guidelines for avoiding plagiarism.

You must complete this coursework individually. Copying code or text from the internet or another student and including it in your assignment without proper attribution constitutes plagiarism.

Undetected plagiarism undermines the quality of your degree by hindering the assessor's ability to evaluate your work fairly and prevents you from learning through the coursework. If plagiarism is suspected, you will be referred to an Academic Misconduct Panel, which may result in academic sanctions.

All assignments will be checked for plagiarism using Turnitin. Therefore, it is important that you correctly reference sources that have informed your work where appropriate

Hyperparameters

For the parameters that are not learnable, you can set them before starting the training.

Format

- Ensure your responses to each question are on the spot as shown in Figure 1. Focus on the specific question and not move away.

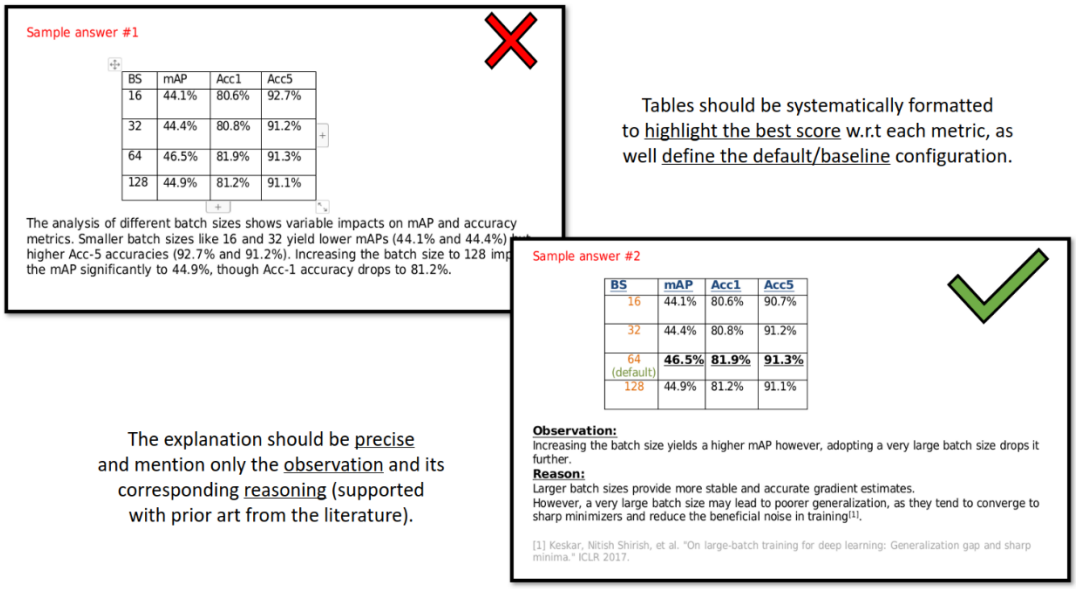

Figure 1: Response example.

Log files

- Submit your own log files from your codebase. You are required to submit a separate log file for each experiment-based question.
- Do not change the log file structure. Each log file is watermarked and unique to each run.
- Name each log file according to the section number and question number: (log_sec{Section_num}_q{Question_num}.txt). For example, the log file generated for an experiment that corresponds to Section 1 Question 2, should be named as “log_sec1_q2.txt”.

Presentation and clarity

In addition to the technical content, the presentation and clarity of your writing will be assessed.

Model performance metrics

While different metrics are available, it is recommended that you prioritise mAP when selecting the best-performance models.

Start your assignment

Write your assignment in the designated space provided below. Each of the sections outlines the tasks required and specifies the information you must include in your report for that section.

Section 1: Familiarity with the code provided (30 marks)

1. Run the code using the default settings. Discuss the training and evaluation process (*mention the loss functions used*) followed by implications of the observed performance using an appropriate metric. (10 marks)

2. Apply another CNN variant (that is not provided in the default settings). Critically discuss and contrast the results with what observed in question 1 above. (10 marks)

3. Apply one more neural network architecture not from the same family as the above questions. Critically discuss and contrast the results with what observed questions 1 and 2 above. (10 marks)

Note:

For Questions 2 and 3, ensure the new architecture is explicitly specified in your shell script command if you are using one from the codebase.

Start writing here:

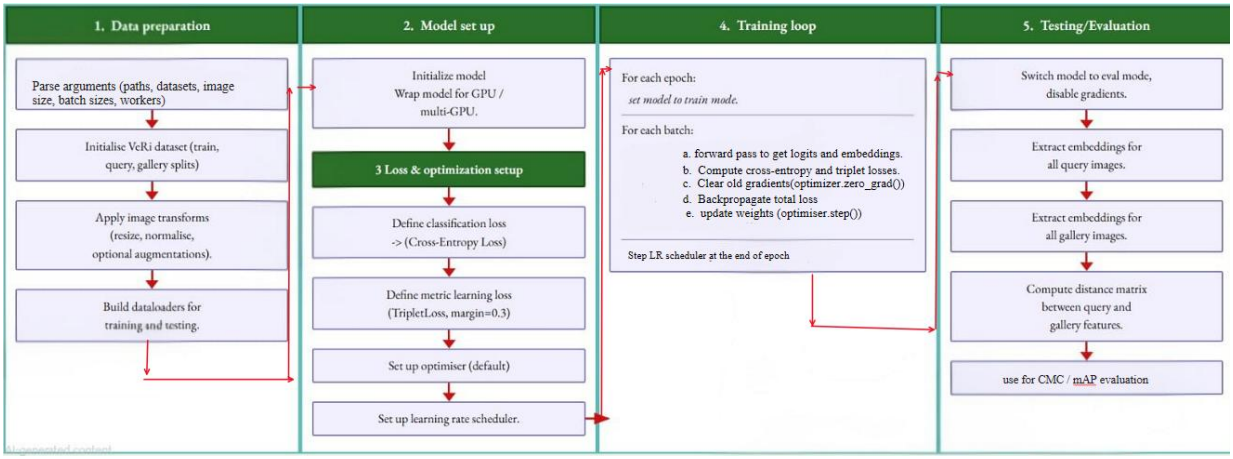

Figure 1:End-to-end training and evaluation pipeline for the vehicle re-identification model, illustrating data preparation, model setup, loss and optimisation configuration, iterative training loop, and final evaluation

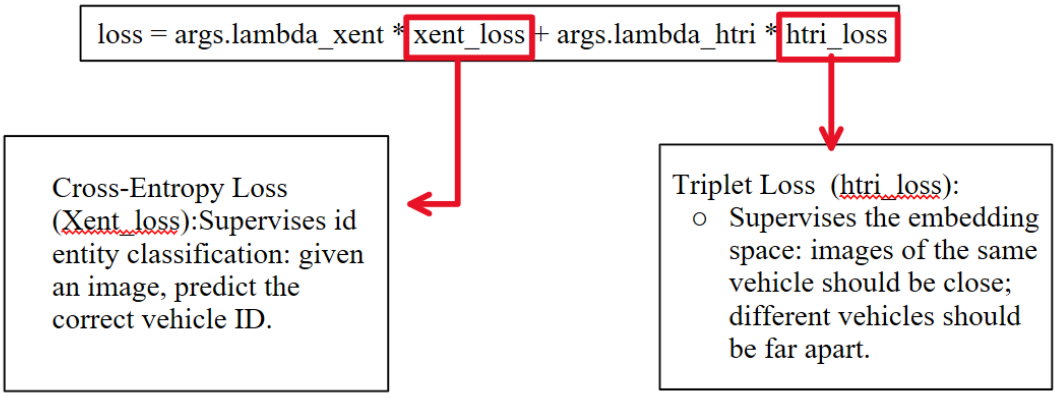

Figure 2: Breakdown of training loss into cross-entropy and triplet terms

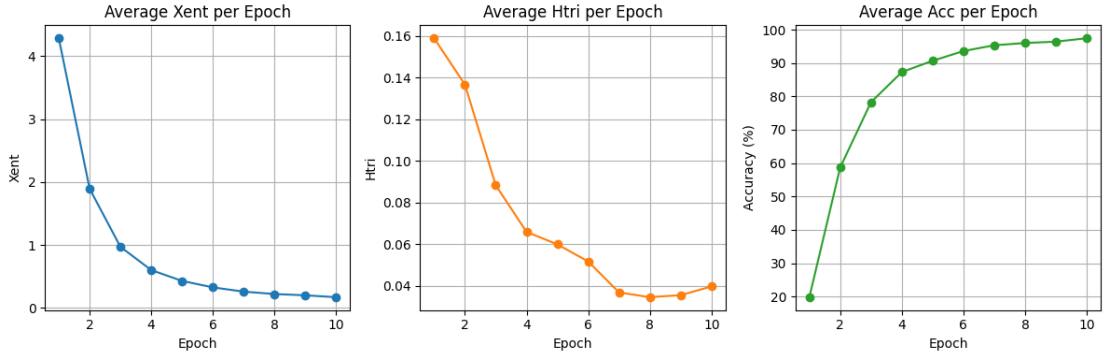

Figure 3:Summary of training behaviour, with cross-entropy loss, triplet loss, and accuracy plotted over 10 epochs for MobileNetV3-Small as per the default settings in train.sh

1.1 Observation:

The MobileNetV3-Small modal achieved a mAP of **44.1%** and a Rank-1 accuracy of **80.2%**, revealing a significant performance gap between the two metrics. Furthermore, the loss curve shows a steady decline followed by a plateau, with no observable "U-turn".

1.1 Reasoning:

High Rank-1 accuracy suggests that the model is proficient at identifying the most visually similar image from the gallery, whereas a lower mAP indicates a struggle to retrieve the full set of relevant gallery images for a specific query vehicle across different viewpoints and lightings. This performance gap often stems from the limited capacity of lightweight architectures, such as the MobileNetV3-Small [1]. The absence of a "U-turn" in the loss curve confirms that the model has converged effectively; however, the plateau suggests that the model is likely underfitting the complex variance of the dataset, as the small parameter count restricts its ability to minimize the residual error further. Using a stronger backbone (e.g ResNet) might increase representational capacity.

Model Architectures	Map(%)	Rank1(%)
mobilenet_v3_small(default)	44.1	80.2
resnet50	51.8	82.7
resnet50_fc512	**54.1***	**86.7***
resnet18_fc512	53.8	86.2
VGG16	23.5	61.0
vit_b16-veri	22.4	51.1

Table 1: Quantitative comparison of architectures for vehicle re-identification on default parameters in Train.sh.

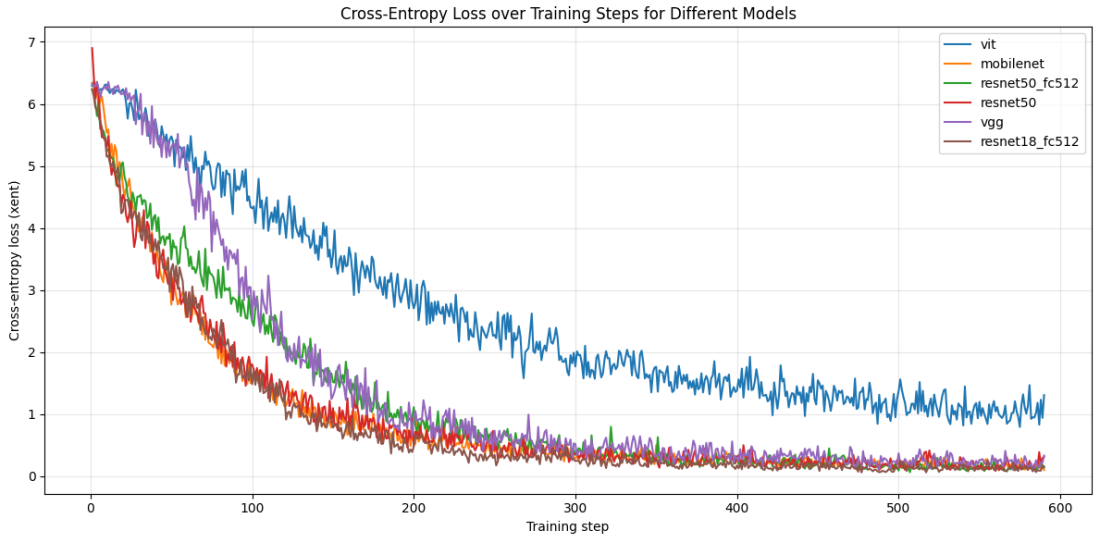

Figure 4: Cross-entropy loss over training steps for different backbone architectures.

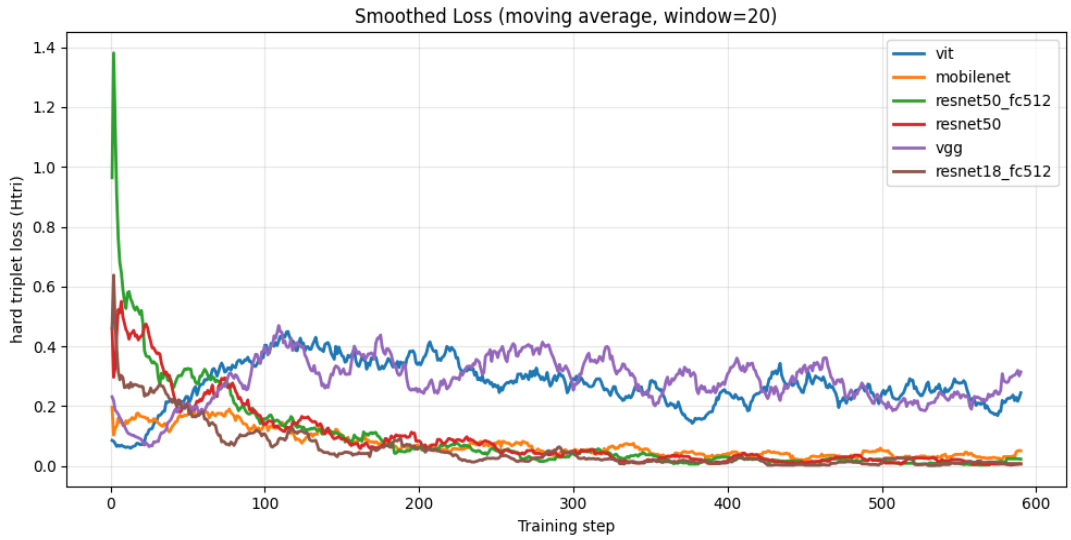

Figure 5: Smoothed hard triplet loss over training steps for different backbone architectures

1.2 Observation:

Among the additional CNN variants tested, ResNet-50_fc512 achieved the highest performance followed closely by ResNet-18_fc512 and ResNet-50. Notably, VGG-16 underperformed. The loss curves further reflect this as ResNet variants converge rapidly and to near-zero xent by step 150, whereas VGG-16 exhibits high variance and slow convergence.

1.2 Reasoning:

The improved performance of this variant is due to the increased depth and the residual learning framework of the ResNet50 architecture, which allows the model to learn more complex feature hierarchies without the degradation associated with deeper networks [2]. The inclusion of the 512-dimensional fully connected layer functions as a bottleneck that encourages the model to extract more discriminative and compact feature embeddings, which is essential for vehicle re-identification where intra-class variance is high [3]

1.3 Observation:

The Vision Transformer (ViT_b16) yielded the lowest performance in terms of both Rank1 and mAP also exhibited a slower convergence rate and maintained noisy cross-entropy loss.

1.3 Reasoning

Transformers rely on global self-attention and lack the local inductive biases of CNNs. As shown in [4], ViT models generally need very large datasets or strong pre-training to learn spatial relationships effectively, which can limit their performance on smaller, specialized datasets such as VeRi [4].

Section 2: Dataset preparation and augmentation experiment (25 marks)

Begin with the default data augmentation setting with random, horizontal flip and Random2DTranslation.

1. Further append on top two additional data augmentation techniques, one at a time e.g., Default + “crop”, Default + “horizontal flip”, and Default + “blurring”. Compare the results with the default configuration in the provided code and discuss the differences in performance. (20 marks)

2. Combine the augmentation techniques from Question 1 to find the best-performing combination. Highlight any improvement or drop in the overall score. (5 marks)

Start writing here:

Augmentation	Map(%)	Rank1(%)
Default augmentation	**54.1***	**86.7***
Default + color augmentation	49.3	85.0
Default + color jitter	53.5	85.4
Default + random erase	53.8	85.2

Table 2: Augmentation comparison on Resnet50_fc512 with default settings in train.sh

2.1 Observation:

Quantitatively, all three individual augmentations resulted in a slight decline compared to the default settings (mAP 54.1%). Random Erasing proved to be the most resilient individual technique with a mAP of 53.8%, while Color Augmentation caused the most significant performance drop, reducing the mAP to 49.3%.

2.1 Reasoning:

The results indicate that the default augmentation already provides a good balance between diversity and fidelity. Adding strong color-based transformations and random erasing increases appearance variability but transformations applied may have been too aggressive, potentially distorting the color-identity features that are critical for distinguishing between vehicles of the same model. Color Jitter, which adjusts brightness, contrast, and saturation, showed a more moderate decline (mAP 53.5%), indicating that while it simulates varying lighting conditions, the intensity of the transformation may need fine-tuning.

Combined Augmentation	Map(%)	Rank1(%)
Default augmentation	54.1	86.7
Default + color augmentation + color jitter	**54.3***	**86.7***
Default + color jitter + random erase	52.7	84.6
Default + random erase + color augmentation	53.1	84.7
Default + random erase + color augmentation + color jitter	52.9	84.4

Table 3: Combined augmentation comparison on Resnet50_fc512 with default settings in train.sh

2.2 Observation:

Among all combinations, default + color_aug + color_jitter achieved the best performance, slightly surpassing the baseline mAP (54.3% vs 54.1%) while maintaining the same Rank-1 (86.7%)

2.2 Reasoning:

Applying two moderate color augmentations together exposes the model to a wider range of illumination and camera color responses, Overall, the experiments suggest that carefully tuned color-based augmentations can marginally improve generalization, whereas spatially destructive augmentations may harm performance [5]

Section 3: Exploration of Hyperparameters (25 marks)

Start with the default learning rate (LR) and batch size (BS).

1. Exploration of Learning Rate (LR). (10 marks)
    a. Experiment with 4 values of LR (in addition to the default value).
    b. Discuss the observed impact of each value on overall performance.
2. Exploration Batch sizes. (10 marks)
    a. Fixing the best LR value from the experiments in question 1 above, test 4 different values of the BS in addition to the default value.
    b. Discuss the impact observed on overall performance.
3. Exploration of the optimizer. (5 marks)
3. Fixing the best Learning Rate value and best Batch Size value from the experiments in Questions 1 and 2, respectively, test with changing the optimizer to SGD, using PyTorch's internal class.
    a. Discuss the impact observed on overall performance.

Start writing here:

Learning rate	mAP (%)	Rank-1 (%)
0.00002	59.7	86.9
0.0001	**66.0**	**91.2**
0.0002	58.6	88.3
0.0003 (default)	54.1	88.3
0.001	37.0	75.5
0.02	9.4	20

Table 4: Learning rate comparison on Resnet50_fc512 with default settings in train.sh

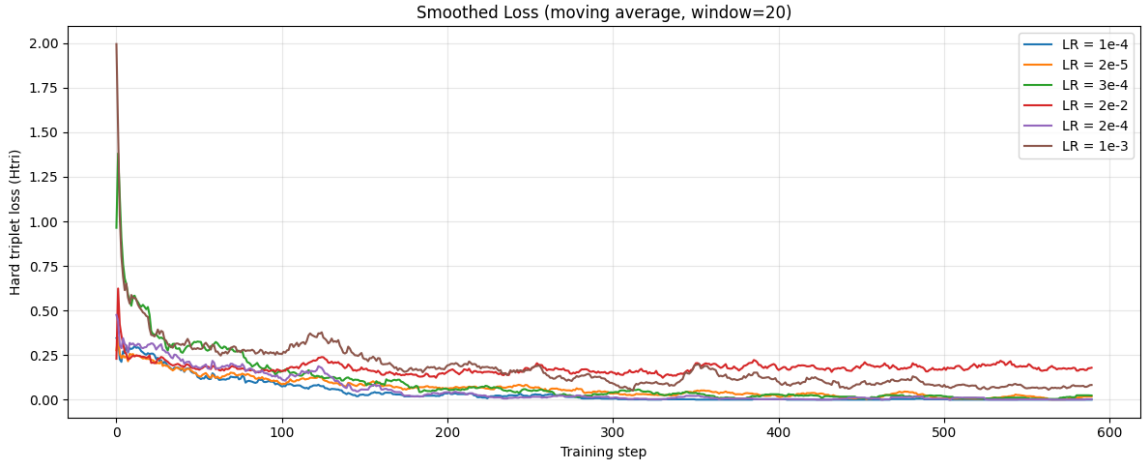

Figure 6:Smoothed Hard Triplet Loss Across Training Steps for Different Learning Rates for Resnet50_fc512

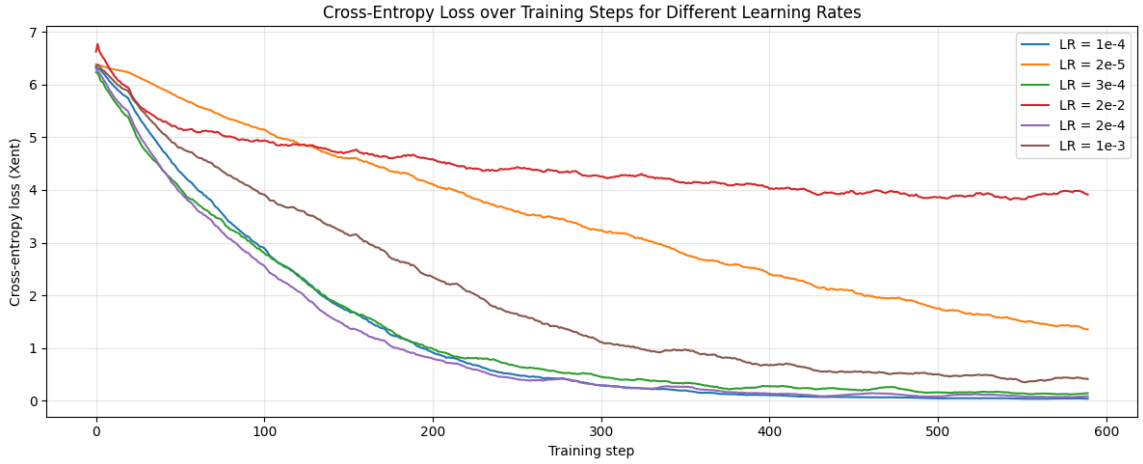

Figure 7:Effect of Learning Rate on Cross-Entropy Loss During Training for Resnet50_fc512

3.1 Observation:

LR 0.0001 emerged as the optimal setting, achieving a peak mAP of 66.0% and Rank-1 accuracy of 91.2%. Reducing the LR below this point (0.00002) led to a slight performance dip, while increasing it beyond the default (0.0003) caused a sharp decline.

3.1 Reasoning:

The learning rate dictates the size of the weight updates if it is too high, the optimizer may "overshoot" the global minimum, leading to the divergence observed at LR 0.02. Conversely, the peak performance at LR 0.0001 suggests that a smaller, more precise step size allows the model to navigate the complex loss landscape of the VeRi dataset more effectively, settling into a flatter and more generalizable minimum. The significant performance drop at higher rates like 0.001 and 0.02 is often attributed to the "exploding gradient" phenomenon or the model getting stuck in sharp, non-generalizable local minima that fail to map query features to the gallery correctly [6].

Batch Size	Map(%)	Rank-1(%)
8	56.7	88.9
16	57.1	88.6
32	63.9	90.2
64 default	**66.0**	**91.2**
128	63.4	89.7

Table 5: Batch size comparison on Resnet50_fc512 with Learning rate of 0.0001 and default settings as per train.sh.

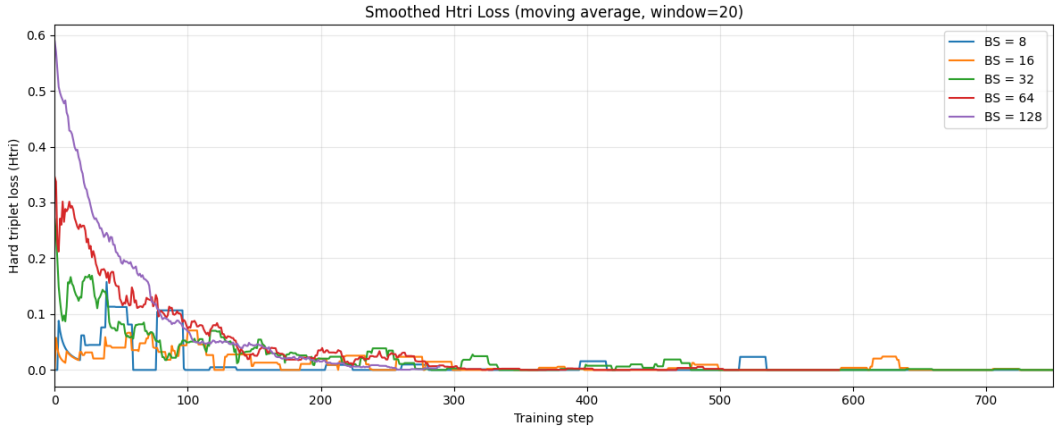

Figure 8 :Effect of batch size on Hard Triplet Loss During Training for Resnet50_fc512 ,lr = 0.0001

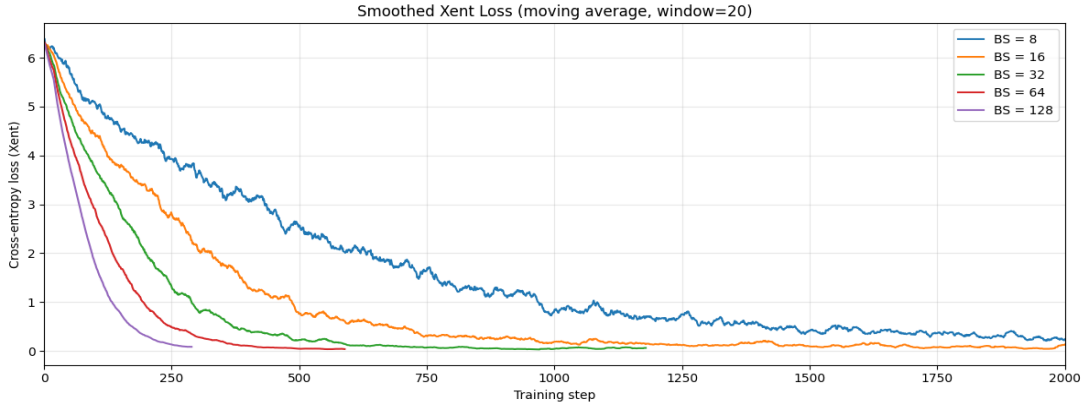

Figure 9: Effect of batch size on Cross Entropy Loss During Training for Resnet50_fc512, lr = 0.0001

3.2 Observation:

Batch size 64 proved optimal, achieving 66.0% mAP and 91.2% Rank-1. Performance was significantly lower for small batches (BS 8 and 16, both ~57% mAP) and declined slightly at BS 128 (63.4% mAP).

3.2 Reasoning:

Batch size 64 provides the ideal balance between gradient stability and model generalization. While small batches suffer from noisy updates that hinder convergence, excessively large batches can lead to a "generalization gap" where the model settles into sharp local minima that perform poorly on test data. This suggests that batch size 64 effectively stabilizes the triplet loss gradients.

Optimizer	Map(%)	Rank-1(%)
SGD	28	56.4
AMS grad (default)	66.0	91.2

Table 6: Optimizer comparison on Resnet50_fc512 with Learning rate of 0.0001, batch size +64 and default settings as per train.sh.

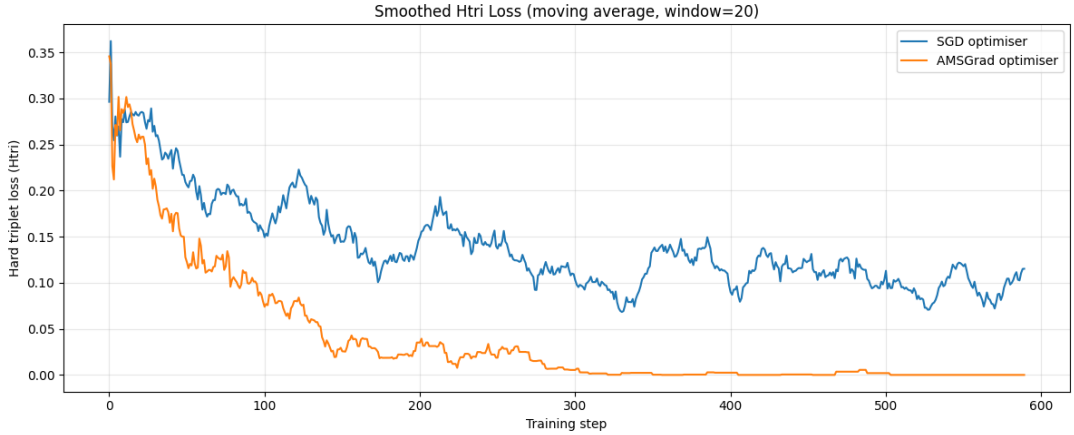

Figure 10: Effect of different optimizers on Hard Triplet Loss During Training for Resnet50_fc512 ,lr = 0.0001, batch size = 64.

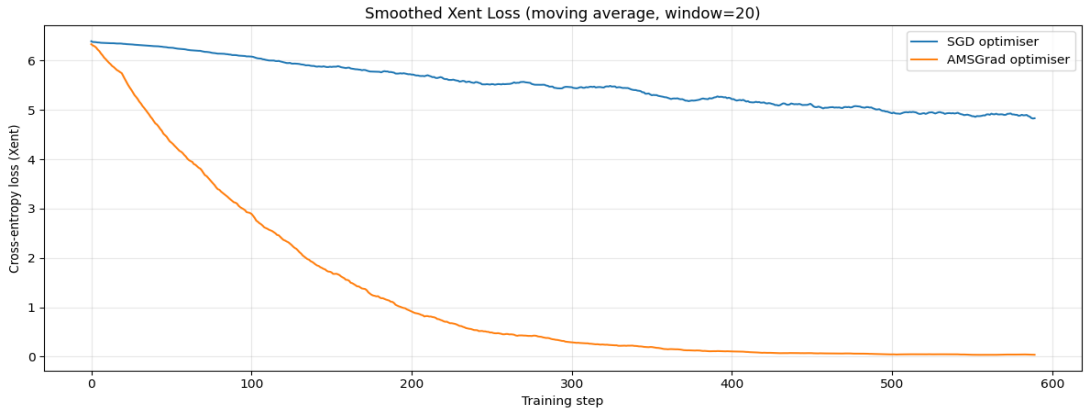

Figure 11: Effect of different optimizers on Cross Entropy Loss During Training for Resnet50_fc512, lr = 0.0001, batch size = 64.

3.3 Observation:

Switching from the default AMSGrad optimizer to SGD resulted in a drastic performance decline: mAP plummeted from 66.0% to 28.0%, and Rank-1 accuracy dropped from 91.2% to 56.4%. Training plots reveal that while AMSGrad converges rapidly toward zero, SGD shows extremely slow descent and high instability, failing to effectively minimize either the cross-entropy and triplet loss within the training period.

3.3 Reasoning:

The disparity highlights the necessity of adaptive gradient methods for complex re-identification tasks. AMSGrad maintains a long-term memory of past gradients to provide stable, parameter-specific updates, which prevents the optimization from stalling in the complex loss landscapes of the VeRi dataset[7]. In contrast, standard SGD lacks these adaptive mechanisms, causing it to struggle with the sparse and noisy gradients produced by the hard triplet loss.

Section 4: Summary on overall hyper-parameter tuning, only need to fill up the table below, no more text. (10 marks)

Section	Hyper-parameter	Best configuration (obtained)	mAP	Rank-1
1	Architecture	resnet50_fc512	54.1	86.7
2	Data augmentation	Default + color augmentation + color jitter	54.3	86.7
3.1	Learning rate	0.0001	66.0	91.2
3.2	Batch size	64 (default)	66.0	91.2
3.3	Optimizer	AMS grad (default)	66.0	91.2

References:

[1] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 1314–1324.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.

[3] L. Zheng, Y. Yang, and A. G. Hauptmann, "Person re-identification: Past, present and future," arXiv preprint arXiv:1610.02984, 2016.

[4] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021

[5] L. Zheng et al., “Improving Person Re-identification by Camera-aware Invariance Learning and Cross-camera Data Augmentation,” CVPR, 2018.

[6] N. S. Keskar, D. Mudigere, J. Jorge, M. Mudigere, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.

[7] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.

Rubric

Section 1: Familiarity with the code provided (30 marks)

Task 1.1: Running and understanding the code (10 marks)

- **Understanding the code execution**
  - *Criteria*: Clear understanding and explanation of the code flow and processes.
  - *Indicators*: Detailed description of data loading, model initialisation, training loop, and evaluation steps.

(5 marks)

- **Metric explanation and performance analysis**
  - *Criteria*: Appropriate choice and understanding of the evaluation metric(s) (e.g., accuracy, mAP).
  - *Indicators*: Correct calculation and interpretation of the metric values. Discussion on what these values imply about the model's performance.

(3 marks)

- **Discussion of observed performance**
  - *Criteria*: Insightful discussion on the observed results.
  - *Indicators*: Analysis of performance patterns, potential overfitting/underfitting issues, and suggestions for improvement.

(2 marks)

Task 1.2: Applying a different CNN variant (10 marks)

- **Implementation of a new CNN variant**
  - *Criteria*: Correctly modifying the code to implement a different CNN architecture.
  - *Indicators*: Model architecture is different from the provided one, and code runs without errors.

(4 marks)

- **Comparison and critical analysis**
  - *Criteria*: Critical comparison between the original and new CNN results.
  - *Indicators*: Use of quantitative results to support comparison, discussion on potential reasons for performance differences.

(6 marks)

Task 1.3: Applying another neural network architecture (10 marks)

- **Implementation of a new neural network**
  - *Criteria*: Correctly implementing a neural network architecture different from those in Tasks 1.1 and 1.2.
  - *Indicators*: Model architecture is distinct, and code runs correctly.

(4 marks)

- **Comprehensive comparison and analysis**
  - *Criteria*: Deep comparative analysis between all three architectures.
  - *Indicators*: Insightful discussion on the strengths and weaknesses of each model, based on quantitative results and qualitative observations.

(6 marks)

Section 2: Dataset preparation and augmentation experiment (25 marks)

Task 2.1: Augmentation techniques (20 marks)

- **Implementation of additional augmentations**
  - *Criteria*: Successful addition of two new augmentation techniques.
  - *Indicators*: Correct application and explanation of chosen techniques, code runs without errors.

(8 marks)

- **Performance comparison and analysis**
  - *Criteria*: Comparative analysis of the impact of each augmentation setting.
  - *Indicators*: Clear explanation of performance differences between default and new settings, supported by quantitative results.

(8 marks)

- **Discussion on results and insights**
  - *Criteria*: Insightful interpretation of how augmentations affected model performance.
  - *Indicators*: Discussion on potential reasons for performance changes and implications for model robustness.

(4 marks)

Task 2.2: Best-performing augmentation combination (5 marks)

- **Combining techniques and experimentation**
  - *Criteria*: Correct implementation of combined augmentation techniques.
  - *Indicators*: Code successfully combines the techniques and executes without errors.

(3 marks)

- **Analysis of improvement or decline**
  - *Criteria*: Detailed discussion on whether the combined augmentation improved performance.
  - *Indicators*: Use of appropriate metrics to highlight any performance change, supported by logical reasoning.

(2 marks)

Section 3: Exploration of hyperparameters (25 marks)

Task 3.1: Learning rate exploration (10 marks)

- **Experimentation with different LR values**
  - *Criteria*: Correct experimentation with three additional LR values.
  - *Indicators*: Clear documentation of the chosen LR values and implementation.

(4 marks)

- **Performance analysis of each LR**
  - *Criteria*: Thorough analysis of the effect of each LR value on performance.
  - *Indicators*: Use of metrics to compare performance across LR values, discussion on the reasons behind observed changes.

(6 marks)

Task 3.2: Batch size exploration (10 marks)

- **Experimentation with different BS values**
  - *Criteria*: Correct experimentation with three additional BS values.
  - *Indicators*: Clear documentation of the chosen BS values and implementation using the best-performing LR from Task 3.1.

(4 marks)

- **Performance analysis of each BS**

- *Criteria*: Detailed analysis of the impact of each BS value on performance.
- *Indicators*: Use of metrics to compare performance across BS values, discussion on the reasons behind observed changes.

(6 marks)

Task 3.3: Optimizer exploration (5 marks)

- **Experimentation with different optimizers**
  - *Criteria*: Correct experimentation with an additional optimizer.
  - *Indicators*: Clear documentation of the chosen optimizer and implementation using the best-performing LR and BS.

(2 marks)

- **Performance analysis of each**
  - *Criteria*: Detailed analysis of the impact of optimizer on performance.
  - *Indicators*: Use of metrics to compare performance across optimizers, discussion on the reasons behind observed changes.

(3 marks)

Section 4: Completing the summary Table (10 marks)

- **Consistency with Experimental Results**
  - *Criteria:* The hyperparameters listed must align with findings from previous sections. Each for **2 marks**.
  - *Indicators:* No contradictions between reported values and earlier discussion.

Report writing and presentation (10 marks)

- **Figures should be well-presented, with readable axes and labels.**

(2 marks)

- **The writing should be clear, grammatically correct, and easy to follow.**

(6 marks)

- **Tables are used when discussing results involving multiple hyperparameters values**
  - *Criteria*: Clarity, structure, and depth of the written report.
  - *Indicators*: Logical flow, clear explanation of methods and results, proper use of figures/tables to support the analysis.

(2 marks)

Total: 100 marks